# Machine Learning Pipeline Workflow

**Goal:** Provide a standardized, end-to-end workflow for building, evaluating, and deploying Machine Learning models. This framework applies to both Supervised and Unsupervised learning scenarios.

## 1. Pre-Modeling & Data Splitting

- **Data Leakage Prevention:** Ensure no information from the test set influences the training process.
- **Train/Validation/Test Split:** Typically 70/15/15 or 80/10/10. Use stratification for imbalanced classification tasks.
- **Cross-Validation:** Use K-Fold Cross-Validation for robust performance estimation, especially on smaller datasets.

## 2. Model Selection Guide

| Learning Type | Algorithm | Best Used For / When to Choose |
|---|---|---|
| Supervised: Regression | Linear Regression | Baseline model; when a linear relationship is assumed. |
| Supervised: Regression | Ridge / Lasso / ElasticNet | When handling multicollinearity or needing feature selection. |
| Supervised: Regression | Random Forest Regressor | Non-linear data; handles outliers well; robust baseline. |
| Supervised: Regression | XGBoost / LightGBM Regressor | High performance; large datasets; winning Kaggle models. |

| Learning Type | Algorithm | Best Used For / When to Choose |
|---|---|---|
| Supervised: Classification | Logistic Regression | Baseline model; probability estimation is required; linearly separable classes. |
| Supervised: Classification | Decision Trees | When high interpretability is needed (e.g., explaining rules to stakeholders). |
| Supervised: Classification | Random Forest Classifier | Excellent general-purpose model; handles missing values and categorical data well. |
| Supervised: Classification | Support Vector Machines (SVM) | High dimensional spaces; complex decision boundaries (using kernels). |
| Supervised: Classification | K-Nearest Neighbors (KNN) | Instance-based learning; good for simple clustering/classification, but computationally heavy at inference. |
| Unsupervised: Clustering | K-Means | Customer segmentation; assumes spherical clusters. |
| Unsupervised: Clustering | DBSCAN | Finding clusters of arbitrary shape; excellent for anomaly/outlier detection. |
| Unsupervised: Dimensionality | PCA (Principal Component Analysis) | Reducing feature space while retaining variance; pre-processing for other models. |

## 3. Model Training & Hyperparameter Tuning

- **Baseline Model:** Always train a simple, un-tuned model first to establish a performance benchmark.
- **Grid Search (GridSearchCV):** Exhaustive search over a specified parameter grid. Best for small parameter spaces.
- **Random Search (RandomizedSearchCV):** Randomly samples from a parameter space. Faster and often more efficient than Grid Search for large spaces.
- **Bayesian Optimization (e.g., Optuna):** Advanced tuning that learns from previous trials to find the optimal hyperparameters efficiently.

## 4. Model Evaluation Metrics

Select metrics based on the business objective, not just statistical accuracy.

| Task | Metric | When to Focus on it |
|---|---|---|
| Regression | RMSE (Root Mean Squared Error) | When large errors are particularly undesirable. |
| Regression | MAE (Mean Absolute Error) | When all errors should be weighted equally (robust to outliers). |
| Regression | R-Squared | To explain the proportion of variance captured by the model. |
| Classification | Accuracy | Only if classes are perfectly balanced. |
| Classification | Precision | When the cost of False Positives is high (e.g., spam detection). |
| Classification | Recall (Sensitivity) | When the cost of False Negatives is high (e.g., disease detection, churn). |

| Task | Metric | When to Focus on it |
| --- | --- | --- |
| Classification | F1-Score | When you need a balance between Precision and Recall, especially on imbalanced datasets. |
| Classification | ROC-AUC | Evaluating the model's ability to distinguish between classes across different thresholds. |

## 5. Explainability (XAI)

- **Feature Importance:** Use built-in tree-based feature importance to see global drivers.
- **SHAP Values:** Use SHAP for complex models (like XGBoost or neural networks) to provide granular, instance-level explanations of why a prediction was made.

## 6. Deployment & Exporting

- **Model Serialization:** Save the final trained model and any preprocessing pipelines (scalers, encoders) using Joblib or Pickle.
- **Pipeline Object:** It is highly recommended to export an entire scikit-learn Pipeline rather than just the model to ensure new data undergoes the exact same transformations.
- **API Readiness:** Structure the output so it can be easily wrapped in a FastAPI or Flask endpoint for production serving.